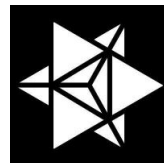


Code Assessment of the Grunt Funds Smart Contracts

Apr 3, 2026

Produced for



by

 **CHAINSECURITY**

Contents

1	Executive Summary	3
2	Assessment Overview	5
3	System Overview	6
4	Trust Model	10
5	Limitations and use of report	15
6	Terminology	16
7	Open Findings	17
8	Resolved Findings	18
9	Informational	31
10	Notes	33

1 Executive Summary

Dear 3F Team,

Thank you for trusting us to help 3F with this security audit. Our executive summary provides an overview of the subjects covered in our audit of the Grunt Fund Integrations according to [Scope](#). This audit will hopefully allow you to form your opinion on the smart contract's security risks.

3F implements additional fund modules for the Grunt protocol, adding Centrifuge and Pareto as new collateral types with asynchronous settlement, enabling users to create on-chain leveraged positions backed by a broader range of asynchronous yield sources.

The most critical subjects covered in this audit are integration correctness and possibilities for adversarial interference. Integration correctness is good. In earlier iterations, the implementations adhered to official Centrifuge and Pareto documentation, but failed to account for edge cases such as instant withdrawal paths, which can lead to lost funds, see [Unhandled Pareto Instant Withdrawals](#) and [Fulfilled Deposit Stuck After cancelRequest Races With approveDeposits](#). These issues have been resolved. Regarding adversarial interference, we identified concerns such as [Spammed Centrifuge Fund Cannot Progress](#), which has since been resolved. Security regarding adversarial interference is good.

The general subjects covered are documentation. The documentation is good. 3F provides extensive documentation for the new integrations, including specifications of the integrated actions and their expected state transitions within the asynchronous lifecycle.

In summary, we find that the codebase provides an improvable level of security.

It is important to note that security audits are time-boxed and cannot uncover all vulnerabilities. They complement but don't replace other vital measures to secure a project.

The following sections will give an overview of the system, our methodology, the issues uncovered, and how they have been addressed. We are happy to receive questions and feedback to improve our service.

Sincerely yours,

ChainSecurity

1.1 Overview of the Findings

Below we provide a brief numerical overview of the findings and how they have been addressed.

Critical -Severity Findings	0
High -Severity Findings	0
Medium -Severity Findings	4
• Code Corrected	3
• Risk Mitigated	1
Low -Severity Findings	7
• Code Corrected	6
• Risk Mitigated	1

2 Assessment Overview

In this section, we briefly describe the overall structure and scope of the engagement, including the code commit which is referenced throughout this report.

2.1 Scope

The assessment was performed on the source code files inside the Grunt repository based on the documentation files. The table below indicates the code versions relevant to this report and when they were received.

V	Date	Commit Hash	Note
1	5 Mar 2026	6e39159125b0dcf2a6732f5885476cbdd4e97093	Initial Version
2	23 Mar 2026	c5b0145665a7bf4d2d45736f5da01363853e7ec1	Fixes
3	1 Apr 2026	12bd55140220e2f6f05353927ba6ba38646ef53f	Final Version

For the solidity smart contracts, the compiler version 0.8.34 was chosen.

The following contracts were in scope for the review:

```
funds/centrifuge/CentrifugeFund.sol
funds/centrifuge/CentrifugeFundFactory.sol
funds/pareto/ParetoFund.sol
funds/pareto/ParetoFundFactory.sol
```

In **Version 2** the following files were added to the scope:

```
request/scripts/SyncDeposit.sol
request/scripts/SyncWithdrawal.sol
request/MorphoFlashLoanRequest.sol
request/MorphoFlashLoanRequestFactory.sol
```

2.1.1 Excluded from scope

Any contracts not explicitly mentioned in the scope like third-party libraries are not in the scope of the review. While the correct integration with third-party protocols like Centrifuge and Pareto is in scope, the protocols themselves are not.

All third-party protocols are out of scope of this review and assumed to work honestly and correctly as documented. In particular the following assumptions were made about the two integrations in scope:

- The external Centrifuge and Pareto contracts are assumed to be correct and are excluded from this assessment.
- The deployed Centrifuge contracts are assumed to be at commit 6b9d36eabee48728486f377ea2766a5cd233c555 of <https://github.com/centrifuge/protocol>.
- The deployed Pareto contracts are assumed to be at commit 767648c656b53369c0d6149b82535e4c6cd9f03c of <https://github.com/Idle-Labs/idle-tranches>.

- The Centrifuge documentation (<https://docs.centrifuge.io/developer/protocol/guides/manage-a-pool/#managing-investment-requests>) and Pareto documentation (<https://docs.pareto.credit/>) are assumed to be correct and complete in regards to possible state changes.
- No reentrancies through Centrifuge or Pareto vault calls are assumed to be possible.
- Pareto and Centrifuge are upgradable. However, it is assumed that there will be no contract upgrade that changes:
 - core functionality Grunt is relying on
 - the possibility for reentrancies when calling these projects

3 System Overview

This system overview describes the **Version 3** of the contracts as defined in the [Assessment Overview](#).

At the end of this report section, we have added subsections for each of the changes according to the versions.

Furthermore, in the findings section, we have added a version icon to each of the findings to increase the readability of the report.

3F implements Grunt, a protocol that unlocks leveraged strategies for asynchronous DeFi assets. Traditional DeFi leverage mechanisms typically rely on the atomicity of flash loans, however, such models are incompatible with assets requiring off-chain or time-delayed settlement. Grunt circumvents this limitation by utilizing short-term bridge liquidity from Bridge Facilitators.

Grunt Funds adds two RWA platform integrations, Centrifuge (JAAA and ACREDX) and Pareto (FalconX Credit Vault), as new collateral types for the Grunt protocol alongside the existing Superstate (USCC) integration.

To scale across these diverse providers, Grunt utilizes a modular architecture where each integration is implemented by a dedicated Fund contract.

3.1 IFund Specification

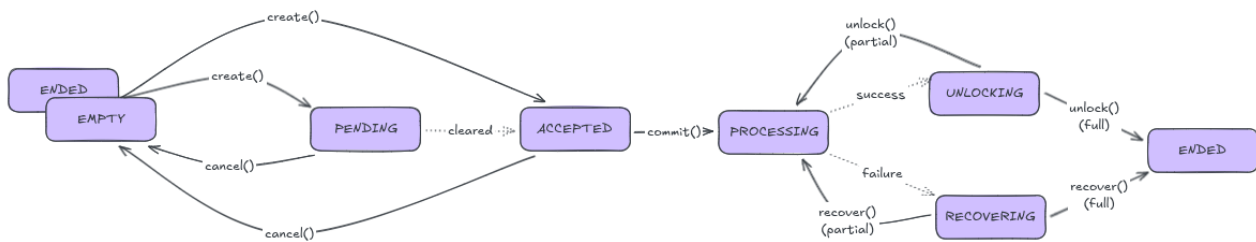
The IFund interface formalizes a standardized state machine for wrapping protocols with asynchronous settlement. To ensure secure and predictable integration with the Facility, all implementations must adhere to the following principles:

1. **Single Order Context:** Each Fund handles at most one active order at a time to prevent accounting overlaps. The content as reflected by a computed orderId is not allowed to change within the fund.
2. **Access Control:** Core state-modifying operations (`create()`, `cancel()`, `commit()`, `unlock()`, `recover()`) are strictly permissioned to the Facilitator. Exceptional administrative actions that transition the state, such as `cancelRequest()` in `CentrifugeFund`, are restricted to the owner or operators.
3. **Valid State Transitions:** Orders must follow a rigid lifecycle, `state(x)` for a given order ID `x` has the following transitions:
 - **create:** EMPTY → PENDING (if internal validation requires asynchronous processing) or ACCEPTED (if the order is immediately eligible for commitment)
 - **cancel:** PENDING or ACCEPTED → EMPTY
 - **commit:** ACCEPTED → PROCESSING or UNLOCKING
 - **unlock:** UNLOCKING → PROCESSING (if partially claimable) or ENDED (if fully claimable)

- **recover:** RECOVERING → PROCESSING (if partially claimable) or ENDED (if fully claimable)

Hence, once `state(x)` returns ENDED for a given order ID `x` it must always return ENDED. `unlock` or `recover` should return ENDED if and only if the `state(x)` also returns ENDED afterwards.

The fund has the following state machine:



Note that the state machine of the Order (as returned by `Fund.state(order)`) has slightly different semantics. While the fund-level state machine cycles back to EMPTY or ENDED and can then process new orders, an individual order will transition to ENDED once and must remain in this state indefinitely.

4. **Asset Definition:** The IFund should define the input asset (`tokenIn`) as the underlying asset () for deposits and the `share()` token for redemptions. Conversely, the output asset (`tokenOut`) is the `share()` token for deposits and the underlying asset () for redemptions.
5. **Asset Transfers:** Token transfers may only occur during specific lifecycle actions to prevent the loss of tokens not snapshotted by Grunt Protocol:
 - `create()` and `cancel()` do not transfer funds.
 - `commit()` transfers `order.input` of `tokenIn` from Facilitator to the Fund and refunds any unprocessed amount.
 - `recover()` returns `tokenIn` to the Facilitator.
 - `unlock()` ensures that `tokenOut` is fully retrieved from the integration and mints the equivalent wrapped `tokenOut` to the Facilitator.
6. **Caveats for view functions:** The IFund view functions provide introspection into the fund and wrapped asset state with some caveats:
 - `totalAssets()` converts the `WrappedAsset` total supply to assets using the vault's conversion rate. It underreports with in-flight orders (committed assets are not yet minted as shares) and overreports when multiple funds share the same `WrappedAsset` (the total supply of the wrapped asset includes shares minted by all funds).
 - `maxRedeem()` returns the fund's `WrappedAsset` balance unconditionally. It does not reflect whether a new order can actually be created, since the fund only supports one active order at a time.
7. **Slippage Protection:** Funds validate `order.output` against a trusted oracle in `create()` and `commit()` and reject orders that deviate by more than `MAX_OUTPUT_DEVIATION`. The check is one-sided: overly optimistic outputs are accepted but may cause the order to remain in PROCESSING until being resolved by the operator.

3.1.1 Centrifuge Fund

The Centrifuge Protocol is designed for tokenizing and distributing financial products using a Hub-and-Spoke architecture. The Hub chain manages accounting and pricing, while Spoke chains handle tokenization and user interactions.

The CentrifugeFund contract acts as an adapter, mapping the IFund state machine to these ERC-7540 compliant vaults. Instances are deployed as beacon proxies via the CentrifugeFundFactory. The native share token of the Centrifuge vault is wrapped into a `WrappedAsset` token to enforce transfer restrictions within the Grunt protocol.

- **Pending:** When `commit()` is called by the Facilitator, the fund submits a request (`requestDeposit` or `requestRedeem`) to the Centrifuge vault, placing the assets into pool escrow on the Spoke chain. The order transitions to the PROCESSING state.
- **Approved & Issued:** The Centrifuge pool manager on the Hub chain approves the batch, locking the assets from cancellation. The manager then issues/revokes shares, which assigns the final execution price. The order remains in the PROCESSING state.
- **Fulfilled:** Once the batch is processed, anyone can call `notifyDeposit` and `notifyRedeem` for any investor address. Then, the assets or shares become claimable (detectable via `maxMint` or `maxWithdraw`), the order moves to UNLOCKING state. Calling `unlock()` finalizes the transfer of the claimable portion to the Facility. If the request was only partially fulfilled, the order transitions back to PROCESSING. Otherwise, it transitions to ENDED.
- **Cancelled:** Before manager approval, the owner or operator can call `cancelRequest()`. The order moves to RECOVERING state, and funds are reclaimed via `recover()` once the cancellation is claimable, transitioning the order to ENDED.
- **Force Ended:** If an order is stuck in PROCESSING or RECOVERING (e.g., due to a grieving deposit or an unresponsive Centrifuge pool), the owner or operator can call `forceEnd()` to forcibly transition the order to ENDED. This is only permitted after all claimable fills and cancel assets have been drained first via `unlock()` and `recover()` respectively. Note that `forceEnd()` is a non-standard state transition that violates the standard IFund lifecycle specification.

It is important to note the nuances of the Centrifuge Vault's asynchronous queueing system. The pool manager may choose to only partially approve the pending requests, any unfulfilled portion stays in the pending queue and remains cancellable. Furthermore, when an operator initiates a cancellation, the action itself is queued asynchronously.

Another distinct feature is that the Centrifuge ERC-7540 vault allows any user to call `AsyncVault.requestDeposit` on behalf of the `CentrifugeFund` controller address. This means external parties can influence the pending deposit state on the vault side without any involvement from `CentrifugeFund` itself. This also allows external parties to donate to the `CentrifugeFund` which is usually prevented in other places.

3.1.2 Pareto Fund

Pareto provides institutional credit vaults based on Idle Finance's `IdleCD0EpochVariant`. These vaults manage liquidity across alternating Buffer Periods and Lending Cycles, allowing synchronous deposits while enforcing asynchronous, epoch-gated withdrawals.

The `ParetoFund` implements the `IFund` interface for Pareto's Credit Vaults. Instances are deployed as beacon proxies via the `ParetoFundFactory`. Similar to other integrations, it enforces Grunt's transfer constraints by utilizing a `WrappedAsset` for the underlying senior (AA) tranche tokens.

Deposit Flow

Deposits in Pareto are synchronous and can occur during either Buffer Periods or active Lending Cycles.

- **Order Commitment:** When `commit()` is called by the Facilitator, the fund executes a deposit into the Pareto vault. Depending on the current Pareto state, this is handled via `depositAA` (during a Buffer Period) or `depositDuringEpoch` (if an epoch is running and this functionality is enabled). In both cases, the underlying assets are transferred, and AA tranche tokens are minted synchronously to the fund. The order transitions to the PROCESSING state.
- **Immediate Execution:** Since the AA tranche tokens are received immediately, the deposit does not need to wait for the epoch to conclude. If the received amount meets expectations, the order transitions directly to UNLOCKING. Otherwise, the order remains in PROCESSING until the owner or operator calls `resolve()` to override the expected output amount.

- **Order Finalization:** Calling `unlock()` finalizes the order. The fund mints an amount of the system's `WrappedAsset` directly to the Facility equivalent to the AA tranche tokens sent to Pareto, transitioning the order to `ENDED`.

Withdrawal Flow

Withdrawals are asynchronous and usually must be requested in advance. The fulfillment timeline depends on the vault's current performance and epoch state.

- **Order Commitment:** When `commit()` is called for redemptions, the fund submits a withdrawal request to the Pareto vault. The system burns the wrapped assets and calls `requestWithdraw`. For normal requests, "receipt strategy tokens" are minted to represent the pending withdrawal. If specific vault conditions are met (such as a significant drop in the underlying strategy's APR), Pareto may automatically route this request to the instant withdrawal path; the `ParetoFund` detects this case and reverts with `InstantWithdrawDetected()` to prevent the order from becoming stuck in `PROCESSING` with no epoch-based fulfillment. The order transitions to the `PROCESSING` state only for normal epoch-gated withdrawals.
- **Asynchronous Processing:** For normal withdrawals, funds typically become claimable at the end of the *following* Lending Cycle, effectively requiring a full cycle's notice. This delay is to give institutional borrowers sufficient time to manage their liquidity, unwind positions, and return capital without forced liquidations. Once the required period has elapsed, the redemptions become claimable and the order moves to the `UNLOCKING` state.
- **Order Finalization:** Calling `unlock()` triggers `claimWithdrawRequest` to retrieve the underlying assets from the Pareto vault, and transfers them to the Facilitator, transitioning the order to `ENDED`.

The `ParetoFund` is designed exclusively for Pareto vaults that do not enable *instant withdrawals*. If an integrated vault ever enables the instant withdrawal path, the `ParetoFund` will reject all redemptions and a new fund implementation would be required to redeem the assets.

3.1.3 Morpho Flash Loan Request

The `MorphoFlashLoanRequest` contract uses Morpho Blue flash loans to atomically execute multi-step deposit and withdrawal operations on a Facility intent. It implements the `IRequest` interface to temporarily register itself as a request on the Facility. Currently this could be used for synchronous deposits into the `ParetoFund`, and could also handle synchronous withdrawals for future fund instances supporting this feature. Each instance is permissionlessly deployed as a beacon proxy via the `MorphoFlashLoanRequestFactory`.

The `MorphoFlashLoanRequest` must be registered as a `Facilitator` in the Facility to perform these batched operations. It implements its own privileged role called `EXECUTOR_ROLE` that is the only address allowed to call function `execute()`, which initiates a flash loan on Morpho Blue. In the callback `onMorphoFlashLoan`, the contract then proceeds to:

1. registers itself as the intent's request via `setRequest()` providing the quorum signatures required by the Facility in `calldata`
2. pulls the flash-loaned tokens into the Facility
3. delegatecalls an owner-whitelisted script that chains Facility fund operations (i.e., `create` → `commit` → `unlock`) and position manager operations (i.e., `depositManager`, `withdrawManager`) which in turn borrow / repay and withdraw from a Morpho Blue market.
4. receives repayment from the Facility, unregisters itself, and approves Morpho to pay back the loan.

The flash loan debt is stored in transient storage and is never explicitly cleared. Consequently, only one `execute()` call can succeed per transaction; a second call will revert because the transient debt is still set.

The contract provides a minimal implementation of the `IRequest` interface, limited to the functions actually called by the Facility: `pullFunds()`, `repay()`, `isRepaid()`, and `asset()`. Even these

deviate from the standard Request contract - for example, `pullFunds` accepts a `calldata` argument but never performs a callback to the puller. All other `IRequest` functions either revert with `NotSupported()` (e.g., `mint()`, `consume()`, `setRepaid()`) or return default values (e.g., `lastMintTimestamp` and `mintToRepaidDelay` return zero). The implementation technically violates the specification of `IRequest`: No user should call these functions outside of a flashloan and expect them to behave as documented. As another addition, the contract has a `rescue()` function that allows the owner to recover stuck tokens.

Scripts

`SyncDeposit` and `SyncWithdrawal` are stateless scripts delegatecalled by `MorphoFlashLoanRequest`. They are expected to be called with Fund integrations that can settle synchronously within `commit()`.

- `SyncDeposit` chains: `create` → `commit` → `unlock` → `depositManager`. Converts assets into wrapped RWA shares via the Fund, then deposits them as collateral into the position manager, borrowing debt tokens to later repay the flash loan.
- `SyncWithdrawal` chains: `withdrawManager` → `create` → `commit` → `unlock`. Repays debt and withdraws collateral from the position manager, then redeems the wrapped RWA shares via the Fund, receiving debt tokens to repay the flash loan.

4 Trust Model

The protocol defines three trust levels for roles and external dependencies:

- **Fully trusted:** A compromised holder of this role can, in the worst case, steal or permanently destroy all user funds through direct administrative action or a combination of parameter changes and contract upgrades. No technical guardrails prevent this.
- **Semi-trusted:** A compromised holder can cause partial fund losses or liveness failures, but by design should not be able to cause a complete loss of user funds through their designated function set alone.
- **Untrusted:** External counterparties whose behavior is strictly constrained and validated entirely on-chain by the protocol's smart contract logic.

4.1 Centrifuge Trusted Roles

The Centrifuge pool's trusted roles (pool admin, fund manager) are **fully trusted**. They control epoch processing, share pricing, and the permissioning system on the vault. A malicious or compromised Centrifuge admin could freeze the fund's assets indefinitely by refusing to process epochs or by revoking the fund's permission on the vault mid-order. They could also manipulate share pricing to extract value, effectively stealing funds from the protocol.

The Centrifuge trusted roles are trusted to provide liveness for the protocol's operation. In particular, they are trusted to process epochs in a regular and timely manner. If epoch processing is delayed or halted, the `CentrifugeFund` remains in `PROCESSING` state, blocking the intent lifecycle and preventing LP claims. However, the owner or operator can invoke `forceEnd()` as a last-resort escape hatch to transition stuck orders to `ENDED`, albeit at the cost of forfeiting any assets still pending in the Centrifuge vault.

The Centrifuge trusted roles are trusted not to make any changes to the vault or pool configuration that would introduce reentrancy attack vectors. The `CentrifugeFund` makes external calls to the Centrifuge vault without reentrancy guards, relying on the vault's own execution model being non-reentrant. Future upgrades need to preserve the immutability of the return values of `share()` and `asset()`.

The Centrifuge pool's trusted roles are trusted to only perform upgrades that will not break the integration. In particular, they are trusted not to extract all value through contract upgrades.

4.2 Pareto Trusted Roles

The Pareto protocol relies on the following external roles, all of which are **fully trusted**:

1. **Curator (Credit Vault Manager)**: The curator is trusted to manage the lending cycles properly. They govern lending cycle parameters, epoch processing, and vault upgrades. A malicious or compromised curator could indefinitely freeze the fund's assets by refusing to stop an epoch or pausing the vault. They could also manipulate interest rates or deploy malicious contract upgrades to drain the underlying assets, effectively stealing funds from the protocol. If epoch processing is delayed or halted, the ParetoFund remains stuck in the PROCESSING state, blocking the intent lifecycle and preventing LP claims.
2. **Borrower**: The institutional borrower is fully trusted to fulfill their repayment obligations. It is assumed that the borrower will bring the expected capital (principal plus accrued interest) back to the credit vault at the end of the lending cycle. A malicious or compromised borrower may refuse to return the borrowed capital, resulting in a direct and unrecoverable loss of the underlying assets.
3. **Keyring Admin**: The Keyring administrator manages the access control whitelist. They are fully trusted not to maliciously revoke any user's or contract's permission. A malicious or compromised Keyring administrator could permanently lock all user assets within the fund by revoking the ParetoFund's permission, causing all interactions with the credit vault to fail as there is no emergency bypass.
4. **Upgradability Admin**: The trusted accounts that can trigger upgrades are trusted not to make any changes to the vault that would introduce reentrancy attack vectors. The ParetoFund makes external calls to the Pareto vault without reentrancy guards, relying on the vault's own execution model being non-reentrant. Furthermore, it is assumed that for a given vault the `strategy()` address, the `AATranche()` address and `token()` address will never change. Additionally, upgrades must not alter core vault functionality in ways that break the ParetoFund's assumptions — for example, enabling third parties to trigger withdrawals for other accounts could be used to freeze the state machine.

4.3 Grunt Bots

The protocol's automated operators (typically backend bots or keeper scripts) are classified as **semi-trusted**. If compromised or acting maliciously, they can cause partial fund losses or halt key protocol operations. By design, a **compromised bot should not be able to cause a complete loss of funds for users or bridge facilitators**.

Facilitator: The Facilitator is a **semi-trusted** automated bot that drives the execution of an intent, including advancing the IFund state machine (`create`, `commit`, `unlock`, `recover`). It is primarily trusted for liveness. If the Facilitator goes offline, assets could remain stuck in Fund contracts, unable to progress through the settlement state machine or return to the Facility.

Executor: The Executor (`EXECUTOR_ROLE` on `MorphoFlashLoanRequest`) is a **semi-trusted** role that can trigger flash-loan-backed operations via `execute()`. The Executor can only invoke scripts that have been previously whitelisted by the owner. If compromised, an Executor could repeatedly initiate flash loan cycles, but each execution is constrained to the whitelisted script logic and will revert atomically if the script fails or if the flash loan cannot be repaid. As the **Executor** calls all functions via the `MorphoFlashLoanRequest` that has received the `Facilitator` role, it can at most do all actions that a normal facilitator could do, except that the order of operations is more limited with them being constrained to the logic of the script and `MorphoFlashLoanRequest` logic.

4.4 Grunt Owners and Admins

Owners and Proxy Admins are **fully trusted**.

- **Proxy Admins & Beacon Owners:** Can atomically upgrade the implementation logic of the Fund, WrappedAsset, and MorphoFlashLoanRequest contracts. A compromised admin could deploy malicious logic to drain assets from any intent or protocol module.
- **Fund Owner:** Can manually adjust order settlement amounts via `resolve()` (in Pareto funds) or initiate asynchronous cancellations via `cancelRequest()` (in Centrifuge fund), or force-end stuck orders via `forceEnd()` (in Centrifuge fund). A compromised owner could cause liveness failures (DoS) by repeatedly cancelling valid requests or force-ending active orders. Furthermore, by maliciously resolving an order, they can artificially lower the required threshold for finalization, causing the Facilitator to accept a loss. Conversely, they could set an incredibly high expected output that prevents the order from passing the checks, permanently halting the state machine.
- **WrappedAsset Owner:** Manages the `ISSUER_ROLE`, which allows specific Fund contracts to mint wrapped assets, as well as the critical transferability roles: `SENDER_ROLE` and `RECEIVER_ROLE`. The `SENDER_ROLE` authorizes an address to send wrapped assets to any recipient, while the `RECEIVER_ROLE` authorizes an address to receive wrapped assets from any sender. If an address lacks the necessary role, the transfer will revert. By revoking these roles, a compromised owner could block the finalization of settled orders by revoking a Fund's issuer privileges.
- **Oracle Configuration:** Oracles for the integrated assets are required. The trusted roles must ensure that the chosen oracles do not provide stale data, update regularly and thereby avoid oracle price jumps of more than 0.5%.
- **Operator:** The Operator is a **fully trusted** role responsible for managing edge cases in the settlement process, such as cancelling pending requests or force-ending stuck orders in CentrifugeFund, or resolving orders in ParetoFund when external settlement amounts differ from initial requests. If compromised, an Operator could cause liveness failures by continuously cancelling valid requests, force-ending active orders in CentrifugeFund (forfeiting pending assets), prevent asset deployment by setting unrealistic expected outputs, or cause significant losses for intents (and thereby users) by unlocking funds prematurely. However, an Operator cannot directly extract funds from the system without colluding with other privileged accounts.
- **MorphoFlashLoanRequest Owner:** Manages the script whitelist via `setScript()` and can recover tokens held by the contract via `rescue()`. Although whitelisted scripts execute via `delegatecall` and can corrupt the contract's own storage (including owner, roles, and facility pointer), the impact on intents is bounded: the MorphoFlashLoanRequest holds at most Facilitator privileges on the Facility, so a compromised owner inherits the same threat surface as a compromised Facilitator — it could cause partial fund losses or liveness failures, but cannot cause a complete loss of user funds. The `rescue()` function additionally allows the owner to withdraw any tokens held by the contract itself (e.g., residual dust from flash loans), which goes above normal Facilitator capabilities.

These roles are expected to be held by the protocol team behind a multisig or governed timelock. No technical guardrail enforces this requirement or delays the execution of owner transactions.

4.5 Other

Underlying Assets: Fully trusted. Assets, like USDC, are trusted to maintain their value and operate securely without unexpected behavior.

Morpho Blue: Fully trusted. The Morpho Blue protocol is assumed to correctly implement flash loans as documented: calling the borrower's `onMorphoFlashLoan` callback with the exact loaned amount, and pulling back the loaned tokens via `safeTransferFrom` immediately upon callback return. No reentrancies through Morpho Blue flash loan calls are assumed to be possible.

4.6 Supported ERC-20

The system handles different ERC-20 tokens. The following ERC-20 tokens are **not** supported:

- Any type of reentrant token (e.g., ERC-777 or other tokens with transfer callbacks).

- ERC-20 where one wei (smallest unit) has a value of more than 0.0001 USD.
- ERC-20 where one wei (smallest unit) has a value of less than $10^{**} - 25$ USD.
- ERC-20 with Fee on Transfer.
- Rebasing ERC-20 where the balance can change without transfers.
- ERC-20 that revert on very large approvals
- ERC-20 with more than 18 decimals
- ERC-20 with multiple entrypoints

4.7 Miscellaneous

The protocol is designed for deployment on Ethereum Mainnet. Before deploying to a different chain, a separate assessment of chain-specific differences and their impact on Grunt's functionality must be performed.

4.8 Changes in Version 2

4.8.1 Terminology

- **Prime Broker → Bridge Facilitator:** The "Prime Broker" term has been renamed to "Bridge Facilitator" throughout the documentation and codebase. The role and functionality remain unchanged.

4.8.2 CentrifugeFund

- **forceEnd() escape hatch:** A new `forceEnd()` function (only `ownerOrRoles(OPERATOR_ROLE)`) allows the owner or operator to force-transition stuck orders from `PROCESSING` or `RECOVERING` to `ENDED`. The function enforces that all claimable fills and cancel assets have been drained first via `_revertIfUnclaimedFills()`. However, if a Centrifuge request is still pending and later fulfilled after `forceEnd()`, those assets are lost.
- **Order ID reuse prevention:** `create()` now checks `endedOrders[orderId]` and reverts with `OrderAlreadyExists()` to prevent reuse of previously ended order IDs.

4.8.3 ParetoFund

- **Instant withdrawal detection and revert:** `commit()` now snapshots `withdrawsRequests(address(this))` before and after calling `requestWithdraw`. If the value does not increase (indicating the CDO routed to the instant withdrawal path), it reverts with `InstantWithdrawDetected()`. This prevents orders from becoming permanently stuck in `PROCESSING`. The previous description of automatic routing to an "accelerated requestInstantWithdraw path" is no longer accurate. In this version, instant withdrawals are now explicitly blocked.
- **lastWithdrawRequest for REDEEM state detection:** `_state()` now uses `lastWithdrawRequest` to detect when epoch-gated redemptions have been fulfilled, unblocking `REDEEM` orders on vaults with 0% APR strategies.
- **Order ID reuse prevention:** Same as other funds.

4.8.4 Request / MorphoFlashLoanRequest (New)

- **MorphoFlashLoanRequest:** A new facilitator contract that uses Morpho flash loans to provide temporary bridge liquidity for a Facility intent. The flow: the executor calls `execute()`, which initiates a Morpho flash loan; in the callback, the contract sets itself as the request on the Facility, pulls the flash loan amount into the Facility via `pullFunds()`, delegatecalls a whitelisted script (e.g., `SyncDeposit` or `SyncWithdrawal`), repays any remaining debt, and approves Morpho to reclaim the loaned funds. The contract separates owner (manages scripts, rescues tokens) and executor (triggers flash loans) roles.
- **SyncDeposit and SyncWithdrawal scripts:** New stateless scripts designed for delegatecall from `MorphoFlashLoanRequest`.

	<code>SyncDeposit</code>	executes	the
<code>create</code>	<code>→ commit</code>	<code>→ unlock</code>	<code>→ depositManager</code>
flow atomically within a flash loan.			
<code>SyncWithdrawal</code>			executes
<code>getIntent</code>	<code>→ withdrawManager</code>	<code>→ create(REDEEM)</code>	<code>→ commit</code>
		<code>→ unlock</code>	

5 Limitations and use of report

Security assessments cannot uncover all existing vulnerabilities; even an assessment in which no vulnerabilities are found is not a guarantee of a secure system. However, code assessments enable the discovery of vulnerabilities that were overlooked during development and areas where additional security measures are necessary. In most cases, applications are either fully protected against a certain type of attack, or they are completely unprotected against it. Some of the issues may affect the entire application, while some lack protection only in certain areas. This is why we carry out a source code assessment aimed at determining all locations that need to be fixed. Within the customer-determined time frame, ChainSecurity has performed an assessment in order to discover as many vulnerabilities as possible.

The focus of our assessment was limited to the code parts defined in the engagement letter. We assessed whether the project follows the provided specifications. These assessments are based on the provided threat model and trust assumptions. We draw attention to the fact that due to inherent limitations in any software development process and software product, an inherent risk exists that even major failures or malfunctions can remain undetected. Further uncertainties exist in any software product or application used during the development, which itself cannot be free from any error or failures. These preconditions can have an impact on the system's code and/or functions and/or operation. We did not assess the underlying third-party infrastructure which adds further inherent risks as we rely on the correct execution of the included third-party technology stack itself. Report readers should also take into account that over the life cycle of any software, changes to the product itself or to the environment in which it is operated can have an impact leading to operational behaviors other than those initially determined in the business specification.

6 Terminology

For the purpose of this assessment, we adopt the following terminology. To classify the severity of our findings, we determine the likelihood and impact (according to the CVSS risk rating methodology).

- *Likelihood* represents the likelihood of a finding to be triggered or exploited in practice
- *Impact* specifies the technical and business-related consequences of a finding
- *Severity* is derived based on the likelihood and the impact

We categorize the findings into four distinct categories, depending on their severity. These severities are derived from the likelihood and the impact using the following table, following a standard risk assessment procedure.

Likelihood	Impact		
	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

As seen in the table above, findings that have both a high likelihood and a high impact are classified as critical. Intuitively, such findings are likely to be triggered and cause significant disruption. Overall, the severity correlates with the associated risk. However, every finding's risk should always be closely checked, regardless of severity.

7 Open Findings

In this section, we describe any open findings. Findings that have been resolved have been moved to the [Resolved Findings](#) section. The findings are split into these different categories:

- **Security**: Related to vulnerabilities that could be exploited by malicious actors
- **Design**: Architectural shortcomings and design inefficiencies
- **Correctness**: Mismatches between specification and implementation

Below we provide a numerical overview of the identified findings, split up by their severity.

Critical -Severity Findings	0
High -Severity Findings	0
Medium -Severity Findings	0
Low -Severity Findings	0

8 Resolved Findings

Here, we list findings that have been resolved during the course of the engagement. Their categories are explained in the [Open Findings](#) section.

Below we provide a numerical overview of the identified findings, split up by their severity.

Critical -Severity Findings	0
High -Severity Findings	0
Medium -Severity Findings	4
• Intent Stuck After Call to forceEnd Code Corrected • Fulfilled Deposit Stuck After cancelRequest Races With approveDeposits Code Corrected • Temporary Virtual Price Inflation in Pareto Enables Collateral Theft Risk Mitigated • Unhandled Pareto Instant Withdrawals Code Corrected	
Low -Severity Findings	7
• forceEnd Violates Specification Code Corrected • CentrifugeFund Recover Violates IFund Spec Code Corrected • Inconsistent Pricing Mechanisms in ParetoFund Deposits Risk Mitigated • Inefficient Struct Packing in Fund Storage Structs Code Corrected • Missing Share Token Transfer Permission Check in CentrifugeFund Code Corrected • Reused Order ID Causes Ambiguous State Code Corrected • Spammed Centrifuge Fund Cannot Progress Code Corrected	
Informational Findings	8
• Incorrect State at Epoch Zero Risk Mitigated • Return Value of isRepaid Is Ambiguous Code Corrected • Cancel Request Blocked by Polluted Claimable Amount Code Corrected • Inconsistent Use of _PENDING_REQUEST Constant Code Corrected • Inconsistent Use of mulDiv in ParetoFund Slippage Guard Code Corrected • Misleading newOrderId Emitted Code Corrected • ParetoFund Violates IFund Spec in Cancel() State Check Code Corrected • Unnecessary Share Token Approvals Code Corrected	

8.1 Intent Stuck After Call to forceEnd

Design **Medium** **Version 2** **Code Corrected**

CS-GRUNTFUND-017

When `CentrifugeFund.forceEnd()` is called, the fund's internal state transitions to `ENDED`. However, the Facility is neither made aware of this change nor can it handle it. The intent keeps storing a reference to the fund and the ended order. This leaves the Facility in an unrecoverable state:

- `resolve()` and `setFund()` revert because `checkNoPendingOrder()` detects a pending order.

- `unlock()` and `recover()` contain logic to delete an ended order, but they call into the fund and revert.

The intent cannot progress to `RESOLVED`, permanently blocking LP claims.

Code corrected:

The Facility's code was updated to be able to remove an order and fund from an intent once the configured fund returns the `ENDED` state for this `orderId`. This is handled by a new function `syncEndedOrder`, which is called in `resolve` and `setFund`.

8.2 Fulfilled Deposit Stuck After `cancelRequest` Races With `approveDeposits`

Design

Medium

Version 1

Code Corrected

CS-GRUNTFUND-001

A race condition between `cancelRequest()` on the `CentrifugeFund` and `approveDeposits()` on the `Centrifuge BatchRequestManager` can leave fulfilled deposit shares permanently unclaimable.

If `approveDeposits()` is called on the `BatchRequestManager` just before the operator calls `cancelRequest()`, the deposit may already be partially or fully approved on the pool side. The `cancelRequest()` guard only checks `maxMint()` at the moment of execution:

```
if (ICentrifugeVault(_vault).maxMint(address(this)) > 0)
    revert LibFundsErrors.PendingClaimableAssets();
```

If the issuance has not happened, then `maxMint() == 0`. The check inside `cancelRequest()` passes and `internalState` is set to `RECOVERING`. Once in `RECOVERING`, only `recover()` can be called, and it transitions exclusively to `RECOVERING` or `ENDED`:

```
bool _hasPendingRecover = _stateHasPendingRecover(order.mode, _vault);
State _newState = _hasPendingRecover ? State.RECOVERING : State.ENDED;
```

There is no path from `RECOVERING` back to `PROCESSING` or `UNLOCKING` at the `internalState` level. The cancelled funds can be claimed through `recover()`, but the fulfilled deposit shares from the concurrent approval cannot be reached through any state transition, leaving them stuck in the vault.

A very similar issue also exists for redemption requests. Then, `maxWithdraw()` could be 0 allowing `cancelRequest()` to succeed and preventing the claim of the redeemed underlyings.

Code corrected:

The new version of `CentrifugeFund` allows `_state()` to return `UNLOCKING` when the current `internalState` is `RECOVERING`. Hence, after `cancelRequest()` has been called, `unlock()` can be called to claim fulfilled returns.

8.3 Temporary Virtual Price Inflation in Pareto Enables Collateral Theft

Security

Medium

Version 1

Risk Mitigated

CS-GRUNTFUND-016

The IdleCDOEpochVariant contract derives virtualPrice from getContractValue(), which includes the contract's underlying token balance. Any underlying tokens donated directly to the contract temporarily inflate virtualPrice until the next call to _skimDonatedAssets(), which forwards the entire balance to the feeReceiver. A PositionManager from Grunt can hold Pareto tranche shares as collateral and relies on their quoted value (derived from virtualPrice) for position health checks.

Therefore, an attacker can donate underlying tokens to the IdleCDOEpochVariant contract to temporarily inflate virtualPrice and thereby the collateral oracle price.

An attack could have the following steps:

1. Attacker observes the mempool for an incoming Facilitator deposit into a PositionManager that uses Pareto shares as collateral.
2. Attacker front-runs the transaction with a donation.
3. The Facilitator's deposit is executed against an inflated collateral valuation. This leaves a position behind which is safe accordingly to the inflated virtualPrice.
4. Subsequently, the attacker triggers an operation that calls _skimDonatedAssets() (such as _deposit(), requestWithdraw(), or depositDuringEpoch()). This resets virtualPrice to its true value. For this step the attacker needs to be whitelisted in the Pareto vault.
5. The newly created position is now unhealthy and eligible for pre-liquidation by the attacker.

```
function _skimDonatedAssets() internal {  
    _transferUnderlyings(feeReceiver, _contractTokenBalance(token));  
}
```

Under specific conditions:

- a highly leveraged PositionManager,
- a narrow gap between safeLTV and pre-liquidation LTV (e.g., 0.89 vs 0.9),
- and a significant share of Pareto tokens held as collateral

the attacker can profitably pre-liquidate the position. Then, the liquidation reward will be bigger than the donation cost.

If the attacker happens to be the feeReceiver of the Pareto Vault, then they always profit from the attack, because they get the donation back as _skimDonatedAssets() transfers all donated tokens to the feeReceiver.

Risk mitigated:

3F states:

```
We will make sure to keep the "safe LTV <-> preLiquidation LTV" spread must  
be significant and the leverage not too high.
```

This together with monitoring that

- not a high percentage of the totalSupply of the Pareto tokens is held as collateral by a PositionManager and

- there are no obvious signs of compromise for the feeReceiver of the Pareto Vault mitigates the risk.

8.4 Unhandled Pareto Instant Withdrawals

Correctness **Medium** **Version 1** **Code Corrected**

CS-GRUNTFUND-002

The ParetoFund handles redemptions by requesting withdrawals from Pareto's Credit Vaults. While most withdrawals are epoch-gated and asynchronous, Pareto supports an instant withdrawal path (e.g., triggered if there is a significant change in the strategy APR).

However, ParetoFund only accounts for normal withdrawals. In `_state()`, the contract only queries the standard withdrawal queue:

```
uint256 _withdrawAmount = IIdleCreditVault(_strategy).withdrawsRequests(address(this));
```

Furthermore, in `unlock()`, it only attempts to claim from the standard withdrawal queue:

```
IIdleCDOEpochVariant($.vault).claimWithdrawRequest();
```

If a withdrawal request is routed to the instant withdrawal path by Pareto's Credit Vault, the ParetoFund is unable to retrieve these assets and the state machine remains stuck in the PROCESSING state.

Note on the audit process: This issue was reported to ChainSecurity by 3F while the audit was ongoing. It was concurrently discovered by ChainSecurity.

Code corrected:

In **Version 2**, `ParetoFund.commit()` detects instant withdrawals by comparing `withdrawsRequests` before and after calling `requestWithdraw()`. If the value did not increase, the CDO routed the withdrawal to the instant path and the function reverts with `InstantWithdrawDetected()`. The specification was changed to only support vaults without instant withdrawals.

8.5 forceEnd Violates Specification

Correctness **Low** **Version 2** **Code Corrected**

CS-GRUNTFUND-018

According to the specification of `ICentrifugeFund` the function `forceEnd()` should:

```
// Reverts with `PendingClaimableAssets` if the vault has claimable fills (maxMint/maxWithdraw)
```

However, the function delegates to `_revertIfUnclaimedFills()`, which skips the check when pending deposit or redeem requests exist. An attacker could dust a small deposit request via the Centrifuge vault and bypass the check, allowing `forceEnd()` to succeed despite claimable fills.

Code corrected:



In **Version 4**, `forceEnd` checks for claimable assets directly instead of delegating the check to `_revertIfUnclaimedFills()`, and reverts if claimable assets exist.

8.6 CentrifugeFund Recover Violates IFund Spec

Correctness **Low** **Version 1** **Code Corrected**

CS-GRUNTFUND-003

The IFund interface specifies that `recover()` should return `PROCESSING` when the recovery is partial:

```
/// @notice Recovers input assets after failed processing.
/// @dev Only callable when state() returns RECOVERING.
/// @param order The order parameters identifying the operation.
/// @return The new state after recovery (going back to PROCESSING if partial, ENDED if full).
/// @return The assets that have been recovered by the receiver.
function recover(Order calldata order) external returns (State, uint256);
```

However, `CentrifugeFund.recover()` transitions to `RECOVERING` on a partial recovery, not `PROCESSING`:

```
bool _hasPendingRecover = _stateHasPendingRecover(order.mode, _vault);
State _newState = _hasPendingRecover ? State.RECOVERING : State.ENDED;
$.internalState = _newState;
```

Code corrected:

The new version of `CentrifugeFund` in its `recover` function returns `PROCESSING` in case of partial recovery:

```
return (_hasPendingRecover ? State.PROCESSING : State.ENDED, _amount);
```

8.7 Inconsistent Pricing Mechanisms in ParetoFund Deposits

Correctness **Low** **Version 1** **Risk Mitigated**

CS-GRUNTFUND-004

For deposit orders, the `ParetoFund` contract performs slippage validation during `create()` by comparing `order.output` against an `_expectedOutput` derived from the vault's `virtualPrice()`:

```
// Slippage guard: reject if expected output deviates too far below the current rate.
uint256 _virtualPrice = IIdleCDOEpochVariant($.vault).virtualPrice($.aaTranche);
uint256 _expectedOutput =
    order.mode == Mode.DEPOSIT ? order.input * 1e18 / _virtualPrice : order.input.mulDiv(_virtualPrice, 1e18);

if (order.output < _expectedOutput) {
    if (_expectedOutput - order.output > _expectedOutput * MAX_OUTPUT_DEVIATION / BPS) {
        revert LibFundsErrors.InvalidOutput();
    }
}
```

This calculation of expected output is inconsistent with the fulfillment logic used in `IdleCDOEpochVariant.depositDuringEpoch()`:

```
function depositDuringEpoch(uint256 _amount, address _tranche) external returns (uint256 _minted) {
    ...
    uint256 interest = _calcInterest(_amount) *
    // the time the depositor actually participates (remaining epoch + full buffer)
    (epochEndDate - block.timestamp + buffer) /
    // the total time baked into the scaled APR (epoch + buffer).
    (epochDuration + buffer);
    ...
}
```

The function `depositDuringEpoch()` mints fewer shares to account for the projected interest income earned by the other depositors up to the deposit time. Calculating expected shares with the `virtualPrice()` yields an incorrectly high `_expectedOutput`.

Since the validation only reverts if

$$\text{order.output} < \text{_expectedOutput} - \lfloor \text{_expectedOutput} \cdot \frac{\text{MAX_OUTPUT_DEVIATION}}{\text{BPS}} \rfloor$$

An incorrectly high `_expectedOutput` makes the check overly restrictive. Further `_state` is using the `order.output` to determine if the withdrawal has fully unlocked:

```
function _state() {
    uint256 _expectedOutput = $.hasResolvedAmounts ? $.resolvedOutput : order.output;
    uint256 _received = $.depositReceived;
    return _received >= _expectedOutput ? (State.UNLOCKING, _received) : (State.PROCESSING, 0);
}
```

An `order.output` set too high could lead to the state machine getting stuck and requiring the operator to manually overwrite the `outAmount` by calling `resolve()`.

For currently known Pareto Funds, the deviation should be less than 5%, which is `MAX_OUTPUT_DEVIATION` at the time of writing. Hence, this should mostly succeed as long as `order.output` is computed correctly and not computed in the way `expectedOutput` is computed.

Risk mitigated:

3F states:

```
order.output won't be computed (off-chain) the exact same way expectedOutput
is. The check in the create() function is only preventing the facilitator to
use unexpected values.
```

8.8 Inefficient Struct Packing in Fund Storage Structs

Design **Low** **Version 1** **Code Corrected**

CS-GRUNTFUND-006

The EVM manages storage in 32-byte slots. Both `ParetoFundStorage` and `CentrifugeFundStorage` place small variables (1-byte enums/bools) after a full 32-byte field, forcing the compiler to allocate a new slot for them.

ParetoFundStorage:

Slot	Field	Size	Unused
0	vault	20 bytes	12 bytes
1	wrappedShare	20 bytes	12 bytes
2	asset	20 bytes	12 bytes
3	aaTranche	20 bytes	12 bytes
4	strategy	20 bytes	12 bytes
5	currentOrderId	32 bytes	0 bytes
6	internalState	1 byte	
	hasResolvedAmounts	1 byte	30 bytes
7	resolvedOutput	32 bytes	0 bytes
8	depositReceived	32 bytes	0 bytes
9	endedOrders	mapping	—

CentrifugeFundStorage:

Slot	Field	Size	Unused
0	vault	20 bytes	12 bytes
1	wrappedShare	20 bytes	12 bytes
2	asset	20 bytes	12 bytes
3	shareToken	20 bytes	12 bytes
4	currentOrderId	32 bytes	0 bytes
5	internalState	1 byte	31 bytes
6	endedOrders	mapping	—

In both cases, relocating these variables to follow the vault address allows the compiler to pack them into Slot 0. This consolidation saves one full storage slot. It reduces gas costs by avoiding an additional SLOAD (2,100 gas) when accessing these related fields in the same execution path.

Code corrected:

The struct elements in `CentrifugeFundStorage` and `ParetoFundStorage` have been reordered for more efficient storage allocation in [Version 4](#).

8.9 Missing Share Token Transfer Permission Check in CentrifugeFund

Design **Low** **Version 1** **Code Corrected**

CS-GRUNTFUND-007

The `CentrifugeFund` checks that it is permissioned on the Centrifuge vault before creating or committing orders:

```
if (!ICentrifugeVault(_vault).isPermissioned(address(this))) {
    revert LibFundsErrors.NotAllowedByFund();
}
```

However, Centrifuge share tokens are restricted ERC-20s that require both sender and receiver to be permissioned for transfers. After a deposit is processed, `unlock()` claims share tokens from the vault and forwards them to the `WrappedAsset` contract via `IWrappedAsset(_wrappedShare).mint()`. If the `WrappedAsset` address is not permissioned on the Centrifuge token, this transfer will revert.

Because the permission gap is not validated during `create()` or `commit()`, the order proceeds through `ACCEPTED` and `PROCESSING` before failing at `unlock()`. At that point assets are already locked in the

vault and can only be recovered through the `async cancelRequest` → `recover` path, requiring operator intervention.

Code corrected:

The new version of `CentrifugeFund` makes sure that both itself as well as the `WrappedAsset` have the necessary permission.

8.10 Reused Order ID Causes Ambiguous State

Correctness **Low** **Version 1** **Code Corrected**

CS-GRUNTFUND-008

The `create()` functions in `CentrifugeFund` and `ParetoFund` do not verify that the computed order ID has not been previously used. If a depositor submits a new order with the same order ID as an order that has been previously ended the `viewer` function `state()` would first check the `endedOrders` mapping and then return state `ENDED`:

```
if ($.endedOrders[_orderId]) {  
    return State.ENDED;  
}
```

Therefore, the return value of `state()` would not match the actual state of that order. As such any integration querying `state()` cannot track the lifecycle of the new order accurately.

Code corrected:

The two funds, in their respective `create` function, now check whether the `_orderId` has been previously used and `revert` if that is the case.

8.11 Spammed Centrifuge Fund Cannot Progress

Security **Low** **Version 1** **Code Corrected**

CS-GRUNTFUND-009

The `Centrifuge ERC-7540` vault allows any user to call `vault.requestDeposit(1, fundAddress, attacker)`

on behalf of the `CentrifugeFund` controller address. This pollutes the vault's `pendingDepositRequest` for the fund with an externally-created 1-wei deposit, and the same applies to `requestRedeem`.

Because `pendingDepositRequest` and `pendingRedeemRequest` are shared counters on the vault side, an attacker can inflate them at negligible cost (1 wei + gas per call). The contract relies on these vault-reported values in `_stateHasPendingRecover` to determine whether all committed input has been processed, so the fund would never observe its pending amount reaching zero and would remain stuck in `PROCESSING` indefinitely.

A grieved fund cannot transition from `PROCESSING` to `ENDED`, blocking the creation of new orders and locking the depositor's workflow until an operator intervenes. The attack is cheap and repeatable, so an attacker could sustain the denial of service for as long as desired.

Code corrected:

In the new version of CentrifugeFund a newly introduced forceEnd function can be called to avoid this spam and proceed to ENDED.

8.12 Incorrect State at Epoch Zero

Informational Version 2 Risk Mitigated

CS-GRUNTFUND-019

In the updated version, ParetoFund._state() queries lastWithdrawRequest rather than withdrawsRequests to detect pending withdrawals, as the latter is not set on the apr0 path in IdleCreditVault. A REDEEM order only transitions to UNLOCKING when lastWithdrawRequest is non-zero:

```
uint256 _lastRequest = IIdleCreditVault(_strategy).lastWithdrawRequest(address(this));
if (_lastRequest > 0) {
    ...
    return (State.UNLOCKING, 0);
}
```

However, epochNumber in IdleCreditVault is initialized to 0 and is only incremented in stopEpoch(), so a withdrawal requested in the very first epoch records lastWithdrawRequest = 0. This would cause an order to remain stuck in PROCESSING.

In practice this requires a fund withdrawal during the vault's very first lending cycle, making it unlikely to occur.

Version 3:

The function ParetoFund.commit() queries IdleCreditVault.lastWithdrawRequest() before and after requesting a withdrawal. It checks whether the value has increased to determine if the withdrawal was as expected (i.e. it did not get routed via the instant withdrawal path, which is unsupported). A withdrawal in epoch zero is non-compliant with this design, as the check cannot distinguish a successful withdrawal in epoch zero from no withdrawal at all.

Risk mitigated:

3F has developed the following process to reduce the risk of this affecting Grunt Funds:

```
We will ensure that an IdleCreditVault is always integrated after the first
lending cycle. For now, the only one we are planning (FalconX) is already past
its first lending cycle.
```

8.13 Return Value of isRepaid Is Ambiguous

Informational Version 2 Code Corrected

CS-GRUNTFUND-021

MorphoFlashLoanRequest.isRepaid() returns _debt() == 0, so after execute() returns, rawDebt remains set in transient storage and isRepaid() returns false despite the loan being repaid.

This does not affect the `execute()` flow, as the Facility queries `isRepaid()` inside the callback before Morpho pulls the tokens. However, the true result after deployment means a Facilitator could unregister an externally set `MorphoFlashLoanRequest` via `setRequest(address(0))`, as the repayment check passes.

Code corrected:

In **Version 3** the raw debt is reset to zero after the request gets repaid.

```
// Clear raw debt so isRepaid() returns true within the same transaction
_setRawDebt(0);
```

Furthermore, 3F clarified that:

```
isRepaid() reflects whether there is outstanding debt, not whether a loan
has historically occurred.
```

8.14 Cancel Request Blocked by Polluted Claimable Amount

Informational **Version 1** **Code Corrected**

CS-GRUNTFUND-023

The Centrifuge ERC-7540 vault allows any user to call

```
vault.requestDeposit(1, fundAddress, attacker)
```

on behalf of the CentrifugeFund controller address. Once such a polluted deposit is processed by the Centrifuge pool, it becomes claimable and increases `maxMint()` for the fund.

The `cancelRequest()` function guards against cancelling orders that have claimable output:

```
if (order.mode == Mode.DEPOSIT) {
    if (ICentrifugeVault(_vault).maxMint(address(this)) > 0)
        revert LibFundsErrors.PendingClaimableAssets();
}
```

If an attacker previously deposited a small amount on behalf of the fund, the resulting claimable shares cause `maxMint()` to return a non-zero value. This makes `cancelRequest()` revert with `PendingClaimableAssets` even though the claimable amount originates entirely from the attacker's pollution, not from the fund's own order. The operator cannot cancel a legitimately pending order until the polluted claimable amount is drained first.

Code corrected:

In the new version of the code, a polluted deposit can no longer block the execution of `cancelRequest`.

8.15 Inconsistent Use of `_PENDING_REQUEST` Constant

Informational Version 1 Code Corrected

CS-GRUNTFUND-010

The contract defines `_PENDING_REQUEST = 0` to represent the Centrifuge convention for the current request and uses it in `cancelRequest()` and `recover()`. However, six call sites in `_state()` and `_stateHasPendingRecover()` pass the literal `0` instead of the named constant:

```
// _state() - uses literal 0
claimableCancelDepositRequest(0, address(this))
claimableCancelRedeemRequest(0, address(this))

// _stateHasPendingRecover() - uses literal 0
pendingCancelDepositRequest(0, address(this))
claimableCancelDepositRequest(0, address(this))
pendingCancelRedeemRequest(0, address(this))
claimableCancelRedeemRequest(0, address(this))
```

These could use `_PENDING_REQUEST` for consistency and to make the semantic meaning clear at every call site.

Code corrected:

The new version of the code makes use of the constant.

8.16 Inconsistent Use of `mulDiv` in ParetoFund Slippage Guard

Informational Version 1 Code Corrected

CS-GRUNTFUND-011

In `ParetoFund.create()` function, the slippage guard calculates the `_expectedOutput` using two different mathematical patterns depending on the order mode:

```
uint256 _expectedOutput =
    order.mode == Mode.DEPOSIT ? order.input * 1e18 / _virtualPrice :
                                order.input.mulDiv(_virtualPrice, 1e18);
```

For the `Mode.DEPOSIT` branch, the calculation uses standard multiplication and division operators, whereas the other branch uses Solady `mulDiv` utility. Using `mulDiv` consistently across both branches could improve code readability and maintainability.

Code corrected:

The new version of the code uses `mulDiv` consistently.

8.17 Misleading newOrderId Emitted

Informational Version 1 Code Corrected

CS-GRUNTFUND-012

`ParetoFund.resolve()` does not update the stored order ID. Instead, it writes resolved amounts to storage and modifies an in-memory copy of order before emitting it:

```
$.hasResolvedAmounts = true;
$.resolvedOutput = output;

order.input = input;
order.output = output;

emit OrderResolved($.currentOrderId, order.toId(address(this)),
    input, output, msg.sender);
```

The `OrderResolved` event specifies its second parameter as the "unique identifier of the new resolved order". However, because the stored order ID is unchanged, the emitted `newOrderId` does not correspond to any on-chain state. Off-chain integrations consuming this event would observe an order ID that cannot get used.

Code corrected:

In [Version 2](#) the `OrderResolved` event no longer has this `newOrderId` argument. Hence, the finding is resolved.

8.18 ParetoFund Violates IFund Spec in Cancel() State Check

Informational Version 1 Code Corrected

CS-GRUNTFUND-013

The `IFund` interface specifies that the `cancel()` function should be callable when the order state is either `ACCEPTED` or `PENDING`:

```
/// @notice Cancels an uncommitted order.
/// @dev Only callable when state() returns ACCEPTED or PENDING.
/// @param order The order parameters identifying the operation.
/// @return The new state of the order after cancellation (EMPTY).
function cancel(Order calldata order) external returns (State);
```

However, `ParetoFund.cancel()` strictly enforces the `internalState` must be `ACCEPTED`:

```
if ($.internalState != State.ACCEPTED) {
    revert LibFundsErrors.InvalidState($.internalState);
}
```

`ParetoFund.create()` always transitions directly to `ACCEPTED`, the `PENDING` state is not reachable. Nevertheless, this implementation violates the `IFund` specification and would break if future updates to `create()` introduced a `PENDING` state.

Code corrected:

The new version of the ParetoFund allows cancelling from PENDING state:

```
if (_internalState != State.ACCEPTED && _internalState != State.PENDING) {  
    revert LibFundsErrors.InvalidState(_internalState);  
}
```

8.19 Unnecessary Share Token Approvals

Informational **Version 1** **Code Corrected**

CS-GRUNTFUND-014

The `commit()` function in ParetoFund approves the tranche token before calling `requestWithdraw` while according to the Pareto documentation and implementation `IdleCDOTranche.burn()` does not require token approval.

Similarly, the `commit()` function in CentrifugeFund approves the Centrifuge vault to pull tranche tokens before calling `requestRedeem()`. Some tranche tokens implement `authTransferFrom` so no approval would be required. Notably, JAAA supports this method, however ARCDX token does not.

Code corrected:

The ParetoFund no longer gives and resets the approval. As mentioned in the description, it is sensible not to make changes in the CentrifugeFund due to version differences.

9 Informational

We utilize this section to point out informational findings that are less severe than issues. These informational issues allow us to point out more theoretical findings. Their explanation hopefully improves the overall understanding of the project's security. Furthermore, we point out findings which are unrelated to security.

9.1 No Validation on Script Payload

Informational **Version 2** **Acknowledged**

CS-GRUNTFUND-020

The `SyncDeposit` or `SyncWithdrawal` script contract passed to `execute()` encodes its own facility address and `intentId`, which are consumed by the script's `run()` function. These are never validated against the requests's facility or the `intentId` specified in `SetRequestParams`. A misconfigured executor could register the contract as a request on one intent while the script operates on a different intent or facility.

In practice this would likely cause the transaction to revert (e.g., the script's facility calls would fail due to missing roles).

Acknowledged:

3F is aware of this behavior and considers it to be in line with their design decisions:

We want to keep this flexibility with the scripts.

9.2 forceEnd Has Limited Protection

Informational **Version 2** **Acknowledged**

CS-GRUNTFUND-022

The `forceEnd()` function checks for claimable cancel assets but does not check for pending cancel requests not yet fulfilled by `Centrifuge`. If a cancellation was submitted but not yet processed, `forceEnd` could still be called and abandon these assets. The operator calling `forceEnd` is fully trusted; however, since only the `CentrifugeFund` can submit cancels, adding this check would be a low-cost safeguard.

Acknowledged:

The 3F has acknowledged the issue but has decided to keep the code unchanged given the late stage of development and the low likelihood of this being an issue in practice.

9.3 Virtual Price Overestimates Redemption Output in Slippage Check

Informational **Version 1** **Risk Accepted**

CS-GRUNTFUND-015

The `create()` function in `ParetoFund` uses `virtualPrice` to estimate the expected output for redemptions. But actual redemption proceeds are reduced by the interest accrued during the buffer period in `IdleCDOEpochVariant._calcInterestWithdrawRequest()`:

```
// calculate total vault interest (they don't get the interest for the buffer period for withdraw requests so
// we scale it back since _calcInterest is scaling the interest with the buffer period),
uint256 totInterest = _calcInterest(getContractValue()) * _duration / (_duration + _buffer);
```

The rationale is that a withdrawing user could exit at the very start of the buffer period, so they should not receive interest in that period. Because `virtualPrice` does not incorporate this discount, the expected output derived from the virtual price is too big.

In practice the mismatch is benign: the output field is not used when claiming withdrawal proceeds. In theory passing a tight, correctly computed output amount to `create()` could cause it to revert the check:

```
if (order.output < _expectedOutput) {
    if (_expectedOutput - order.output > _expectedOutput * MAX_OUTPUT_DEVIATION / BPS) {
        revert LibFundsErrors.InvalidOutput();
    }
}
```

However, the 5% slippage tolerance (`MAX_OUTPUT_DEVIATION`) is far larger than the buffer discount under normal conditions (i.e. regular interest rates and a short buffer period compared to epoch period), so this revert is unlikely to occur in practice.

Risk accepted:

3F has accepted the risk, but has decided to keep the code unchanged.

10 Notes

We leverage this section to highlight further findings that are not necessarily issues. The mentioned topics serve to clarify or support the report, but do not require an immediate modification inside the project. Instead, they should raise awareness in order to improve the overall understanding.

10.1 3F Must Regularly Call Centrifuge's notifyDeposit and notifyRedeem

Note **Version 1**

In Centrifuge, after the fund manager approves a batch and issues or revokes shares, individual investors' fulfilled amounts are not distributed automatically. Instead any user can call `BatchRequestManager.notifyDeposit()` and `BatchRequestManager.notifyRedeem()` for each user to dispatch the tokens.

<https://docs.centrifuge.io/developer/protocol/guides/manage-a-pool/#notifying-vaults-for-fulfillment>

It is not fully clear whether the fund operator is responsible for calling these functions or whether another party (e.g., a relayer or the investors themselves) is expected to do so. Regardless of who is responsible, if neither function is called, an attacker can pollute the deposit queue by submitting many tiny deposit requests. Because `notifyDeposit()` has not been called for those investors, their orders remain queued and inflate `pendingDeposit`, affecting pro-rata calculations for legitimate depositors.

Operators can prevent this by either calling both functions regularly **for all instances of CentrifugeFund** to flush processed orders, or by relying on another trusted party to do so. In either case the responsibility should be clearly assigned so that stale orders do not accumulate and fulfilled funds can be claimed in a timely manner.

10.2 Centrifuge Epoch Approval DoS via Pre-Approval Cancellation Front-Running

Note **Version 1**

The Centrifuge `BatchRequestManager` validates manager-submitted approvals against the current `pendingDeposit` total:

```
uint128 pendingAssetAmount = pendingDeposit[poolId][scId_][depositAssetId];
require(approvedAssetAmount <= pendingAssetAmount, InsufficientPending());
```

Whether a deposit cancellation directly mutates the pending total or if the deposit is queued depends on `_canMutatePending()`:

```
function _canMutatePending(UserOrder memory userOrder, uint32 currentEpoch)
    internal pure returns (bool)
{
    return currentEpoch <= 1
        || userOrder.pending == 0
        || userOrder.lastUpdate >= currentEpoch;
}
```

When the epoch has not advanced since the attacker's deposit, `_canMutatePending()` returns true because `userOrder.lastUpdate >= currentEpoch`. The cancellation directly reduces

`pendingDeposit`, enabling a front-running attack: An attacker deposits, waits for the manager to observe the inflated pending total and submit `approveDeposits`, then front-runs the manager's call with `cancelDepositRequest` to reduce the total pending asset amount below the approved amount. The manager's transaction reverts at the `InsufficientPending` check.

The attacker can repeat this at negligible cost, forcing the manager to retry with corrected amounts each time and wasting gas. While post-approval cancellations are routed to a queue and are not susceptible, the pre-approval window remains exploitable for griefing the epoch approval flow.

This is a known issue in Centrifuge, but could theoretically affect Grunt as approvals could be delayed by this.

10.3 Centrifuge Slippage Check Bypassed When Vault Price Is Unset

Note Version 1

The `create()` function validates slippage by computing an `_expectedOutput` from the vault's current price via `ICentrifugeVault.convertToShares()` (for deposits) or `convertToAssets()` (for redemptions). This ensures that the caller-supplied `order.output` is not unreasonably far below the fair exchange rate.

However, if the vault has no price set yet, `convertToShares()` returns 0, making `_expectedOutput` zero. Because `order.output` is a `uint256`, the condition `order.output < 0` is always false, so the slippage guard is silently skipped for any output value.

As a result, an order can be created and committed against an unpriced vault without any slippage protection. This allows orders with arbitrarily low or mismatched output amounts to pass validation, undermining the idea of the slippage guard.

10.4 No Output Validation in Unlock

Note Version 1

Both `CentrifugeFund` and `ParetoFund` validate `order.output` against current conversion rates in `create()`, but neither contract checks the actual received amount against `order.output` in `unlock()`. For deposits and redemptions which execute asynchronously and not in one atomic transaction, `unlock()` can only claim whatever the underlying vault provides and forward it to the receiver.

The conversion rate at execution time may differ from the rate at creation time. The `order.output` field therefore serves only as a creation-time sanity check: `create()` reverts if the expected output deviates too far from current rates, but no guarantee is made on the actual amount received at fulfillment. In `CentrifugeFund` specifically, this behaves more as a stuck-order guard than true slippage protection, since the rate used by `convertToShares()` / `convertToAssets()` and the fill rate can diverge materially.

The one exception is `ParetoFund.deposit()`, which can execute synchronously via a flash loan. In that case `order.output` does function as an effective slippage check, since request and issuance happen in the same transaction. If the received amount is less than output, the order's state machine does not reach `ENDED`, which would allow the facilitator to revert the transaction.

10.5 Non-standard State Transition During Recovery

Note Version 2

In CentrifugeFund, when the internal state is RECOVERING, the function `_state()` checks for fulfilled shares / assets and returns state UNLOCKING if there are any. This allows function `unlock()` to be called. The `unlock` function can set the internal state back to RECOVERING if cancel assets are pending or claimable:

```
if ($.internalState == State.RECOVERING) {  
  bool _hasPendingRecover = _stateHasPendingRecover(order.mode, _vault);  
  bool _hasPending = _hasPendingRecover || _hasPendingRequest;  
  $.internalState = _hasPending ? State.RECOVERING : State.ENDED;  
}
```

This results in an UNLOCKING → RECOVERING transition, which is not described by the IFund state machine specification.

10.6 Pareto Borrower Default Likely Requires Contract Upgrade

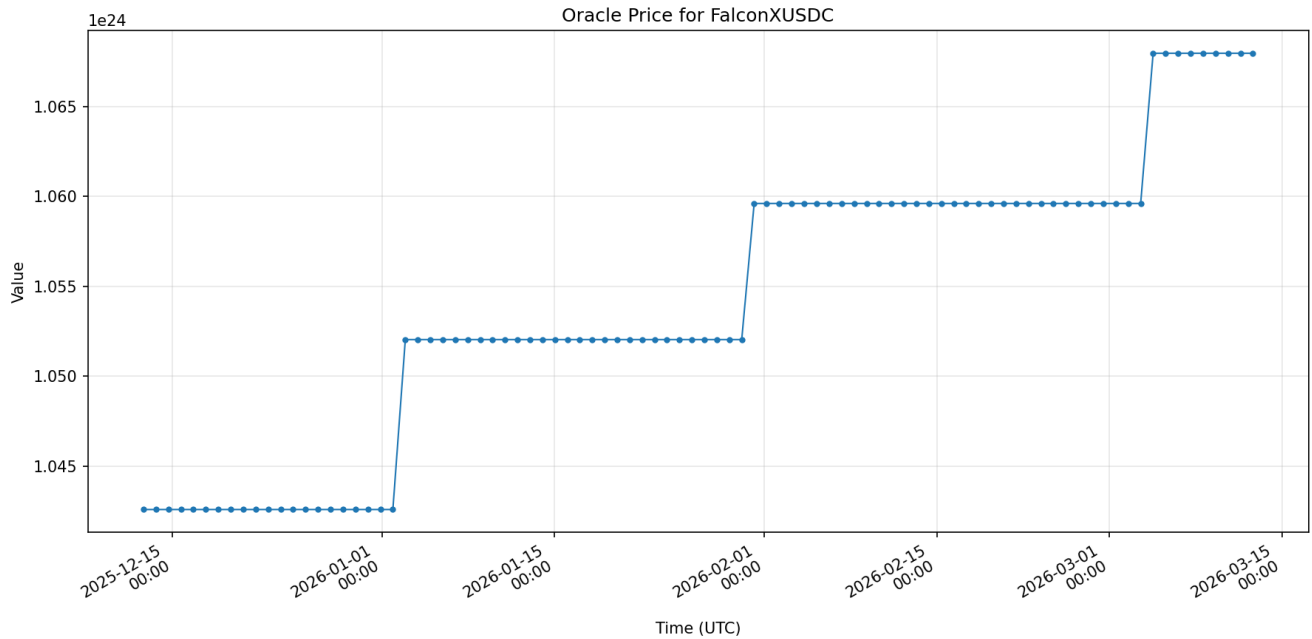
Note Version 1

If a Pareto borrower defaults on the loan then any pending deposit or withdrawal can get stuck. If a fund has a pending withdrawal request, it receives strategy tokens. These tokens are expected to be used as claims on the recovery process. Note that the current state machine is not aware of a default event, i.e. it does not query the `defaulted()` flag on the `IdleCreditVault`. However, there is also no documentation on what an integration could do in this event, so under these considerations the fund might need to get upgraded to resolve the situation.

10.7 Pareto's Virtual Price Is Stale Between Settlements

Note Version 1

Pareto's documentation recommends `IdleCDOEpochVariant.virtualPrice()` for pricing tranche shares. This value is only updated when an epoch is stopped (approximately every 30 days). Between epochs, the price remains stale:



On-chain virtualPrice() for the past 90 days of the FalconXUSDC AA tranche over time, showing the step function at epoch settlement.

This stale pricing can affect three areas of Grunt's core protocol:

A) Performance fees. When virtualPrice() jumps at epoch settlement, PositionManagerBase._accrueFees() computes performance fees on the gross collateral gains without deducting the realized morpho debt interest that was already reflected in lastTotalAssets.

B) Position health. The health check in MorphoBorrowPosition._isHealthy() sees the effective loan-to-value ratio drift steadily above safeLtv throughout the epoch, since collateral value stays constant while debt value grows. Rebalancing operations can be challenging: leveraging up is straightforward since deposits are synchronous given a whitelisted depositor, while leveraging down can be challenging since redeems can take between 30 and 63 days for the rebalancer.

C) Unfair liquidations. A position that is economically healthy can breach liquidationLtv simply because the oracle has not yet reflected the underlying yield. A liquidator calling preLiquidate() at this point seizes collateral at a price that understates its true value, extracting value from depositors.

10.8 Partial Fills Can Leave Residual Dust in Perpetual PROCESSING State

Note Version 1

The Centrifuge protocol supports partial fulfillment of deposit and redeem requests. When a fund manager approves a batch, each pending request is fulfilled proportionally. Unfulfilled portions remain queued for the next batch cycle. For private credit vaults (e.g., ACRDX) partial fills are expected, particularly on redemptions where off-chain liquidity may be constrained. In BatchRequestManager.approveDeposits(), the approvedAssetAmount parameter can be any value up to the total pending amount:

```
uint128 pendingAssetAmount = pendingDeposit[poolId][scId_][depositAssetId];
require(approvedAssetAmount <= pendingAssetAmount, InsufficientPending());
```

Each user gets filled pro-rata: if 90% of the batch is approved, each user receives 90% of their deposit. When many consecutive epochs only fill partially, a small residual amount can remain pending

indefinitely and the CentrifugeFund order never transitions from PROCESSING to ENDED. Operators should monitor for dust amounts stuck in PROCESSING and cancel these orders via `cancelRequest()` to unblock the fund.

10.9 Whitelist Removal Bypasses cancelRequest Guard

Note Version 1

The `cancelRequest()` function in `CentrifugeFund` guards against cancelling orders that have claimable output:

```
if (order.mode == Mode.DEPOSIT) {
  if (ICentrifugeVault(_vault).maxMint(address(this)) > 0)
    revert LibFundsErrors.PendingClaimableAssets();
} else {
  if (ICentrifugeVault(_vault).maxWithdraw(address(this)) > 0)
    revert LibFundsErrors.PendingClaimableAssets();
}
```

However, the underlying `Centrifuge AsyncRequestManager` functions `maxMint` and `maxWithdraw` include a `_canTransfer` check that returns zero when the fund is not on the share token whitelist, regardless of actual claimable amounts.

If the `CentrifugeFund` is removed from the whitelist while it has a `PROCESSING` order, the guard above passes even when there are fulfilled shares or assets. For deposit orders, `cancelDepositRequest` on the vault does not independently check transfer restrictions, so the cancellation succeeds. The fund transitions to `RECOVERING`, but any shares from a partial fulfillment are lost. For redeem orders, the guard also passes, but `cancelRedeemRequest` on the `AsyncRequestManager` independently checks `_canTransfer` and reverts with `TransferNotAllowed`.

As a secondary effect, `_state()` also relies on `maxMint` / `maxWithdraw` for the `PROCESSING` → `UNLOCKING` transition, so the fund remains stuck in `PROCESSING` until re-added to the whitelist.

10.10 forceEnd Can Abandon Committed Assets

Note Version 2

`CentrifugeFund.forceEnd()` was introduced in [Version 2](#) to handle the grieving vector where an attacker dusts `pendingDepositRequest` to keep the fund stuck in `PROCESSING`.

However, with the way it is implemented, an operator could call `forceEnd` right after `commit` has been called. That would essentially abandon all committed assets from the intent's perspective.

The operator can also call `forceEnd()` in other states, when (some) committed assets have not yet been minted into shares by the `Centrifuge` vault. Those assets are then abandoned by the intent. The assets remain in the vault's escrow.

The operator is fully trusted in [Version 2](#) and hence this is not considered a finding. The stranded assets could in theory be recovered via a complicated sequence of `Facilitator` actions that require that no other intent would use this contract in the meantime.